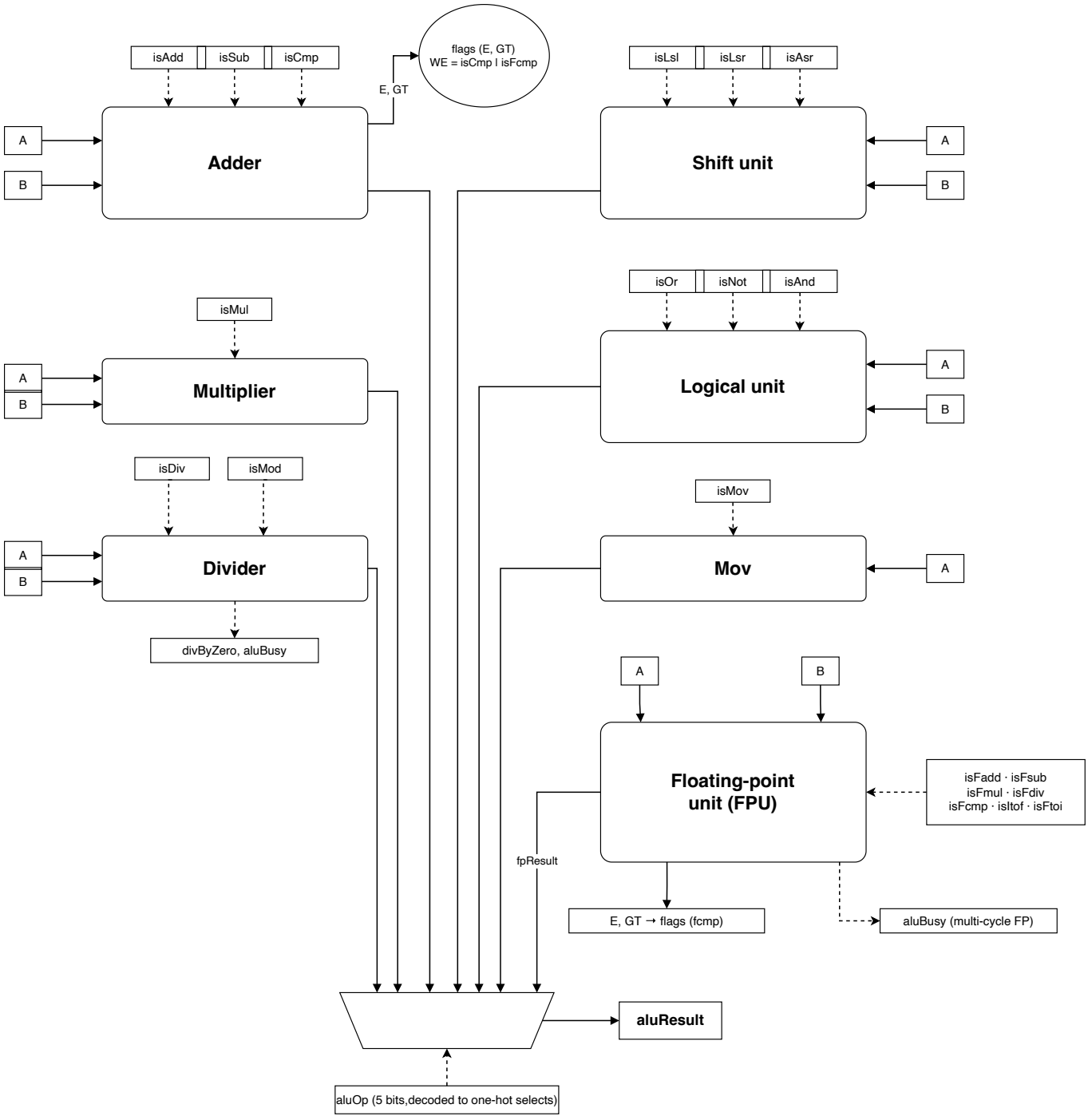
ALU

Overall Structure



Adder / Subtractor

Implemented using the Kogge-Stone parallel-prefix algorithm, as it gives the minimum carry delay (5 levels for 32 bits) with low fanout, at the cost of extra area.

Advantage:

Kogge-Stone is the fastest parallel-prefix adder: only 5 levels for 32 bits, and every node drives only 2 others. The cost is more area and wiring.

working:-

Pi = Ai ^ Bi

Gi = Ai & Bi

Pi and Gi is precomputed

Intermediate signals

CPi:j = (Pi:k + 1)&(Pk:j)

CGi:j = Gi:k + 1 | (Pi:k + 1 & Gk:j)

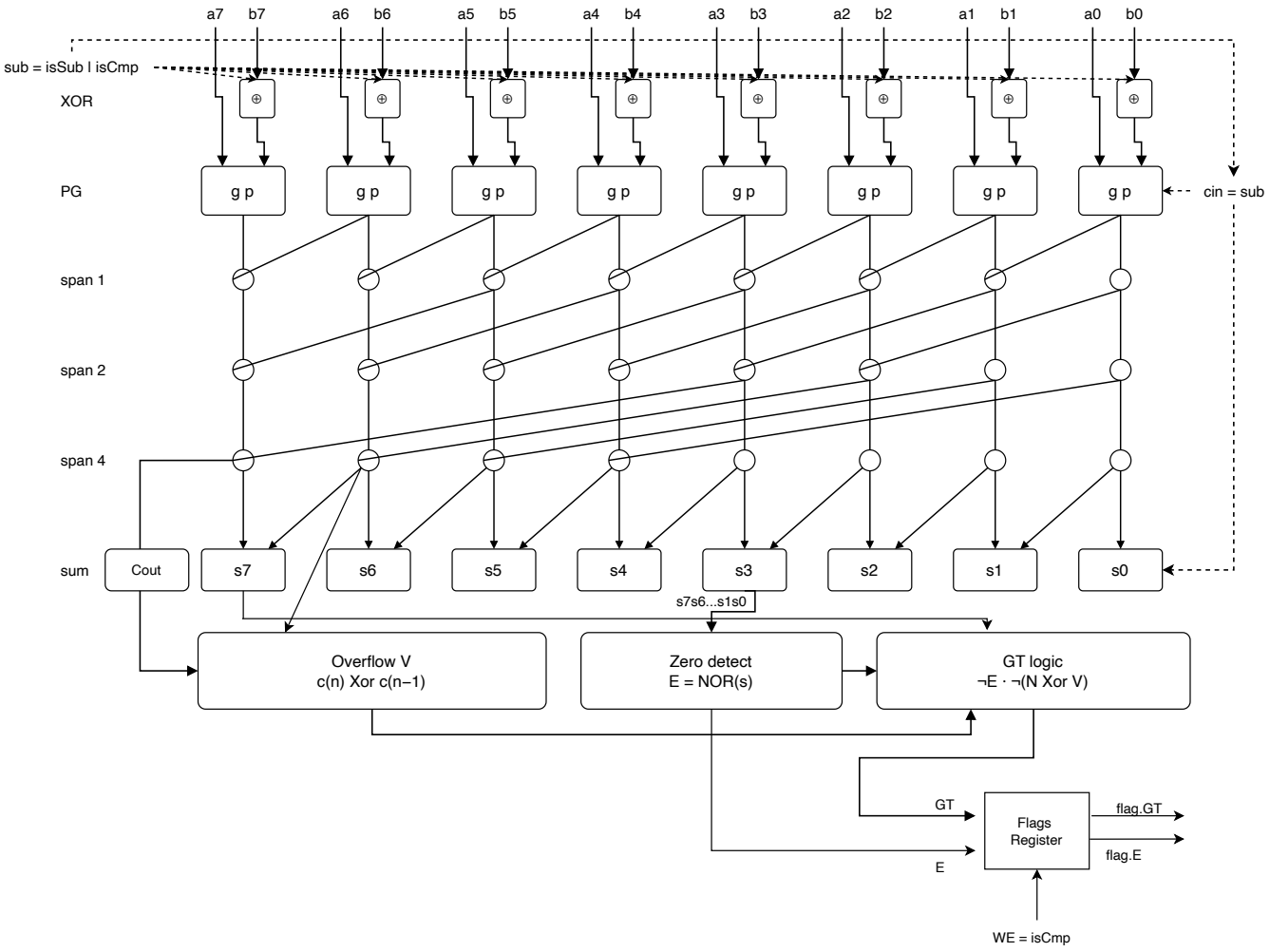
Post-processing

$G_0 = G_0 \mid (P_0 \,\&\, Cin)$

$C_i = G(i-1 : 0)$

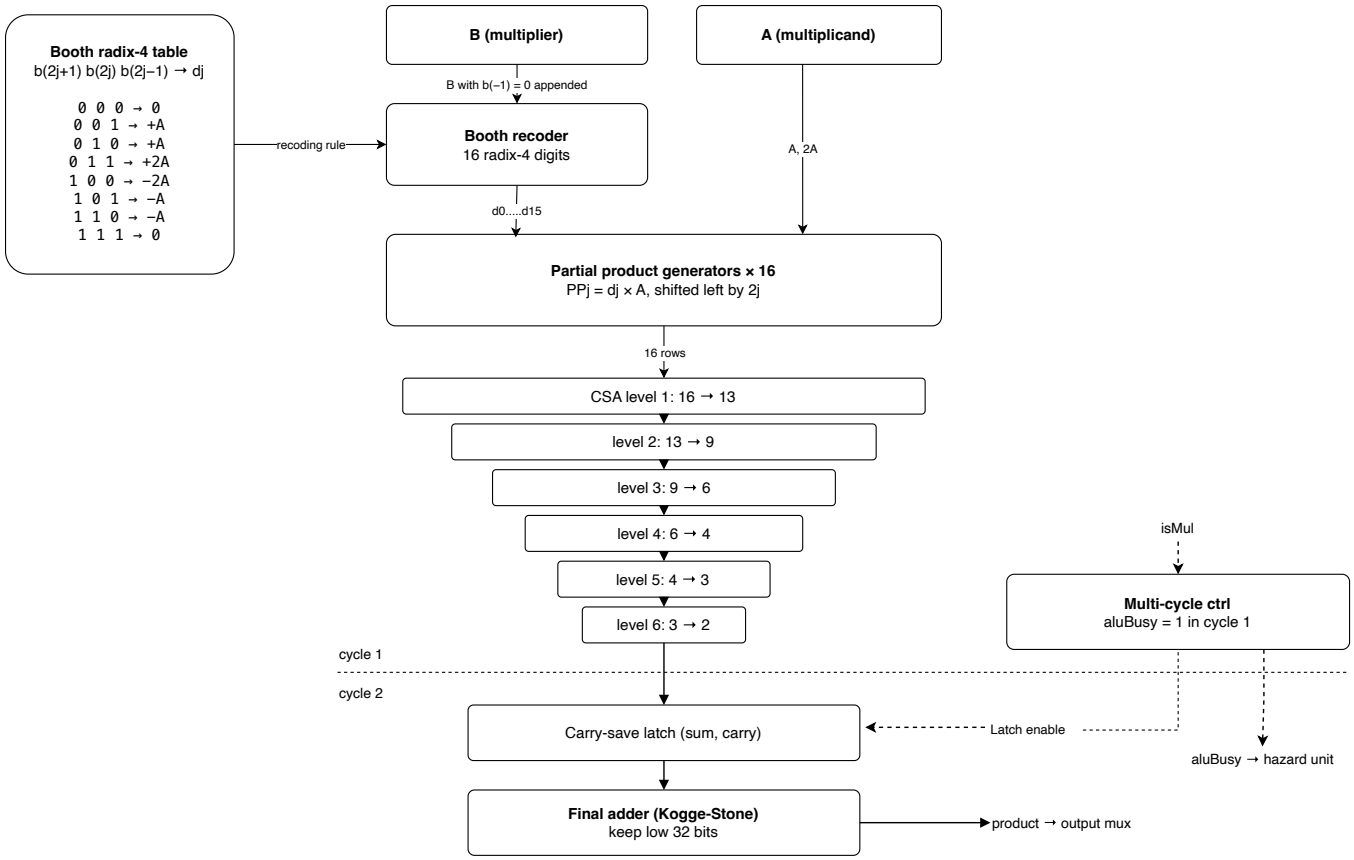
$S_i = P_i \oplus C_i$

Kogge-Stone adder/subtractor



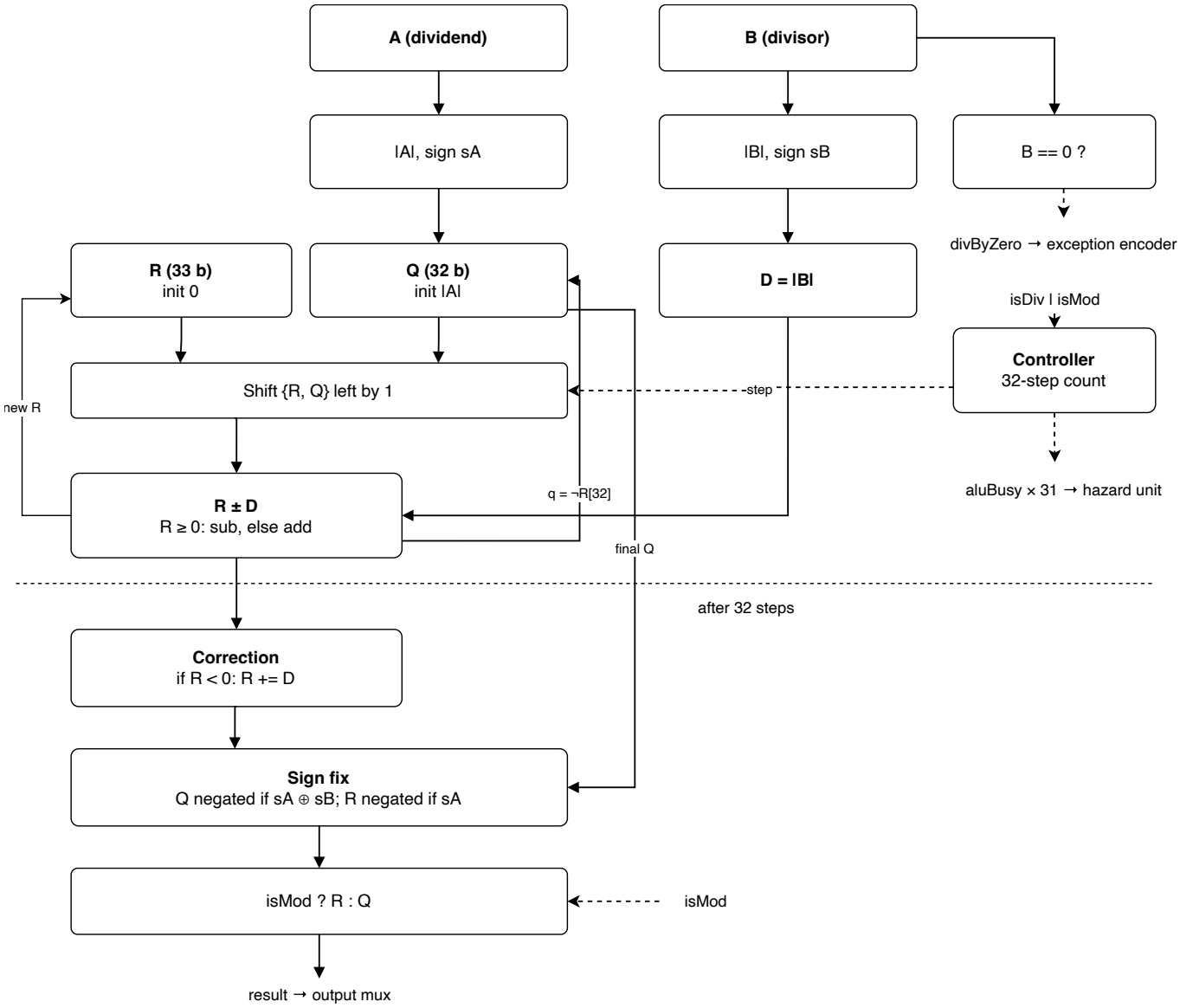
Multiplier

Implemented using radix-4 Booth encoding with a Dadda tree. Booth recoding halves the partial products (32 → 16), and the Dadda tree reduces them to 2 rows in 6 levels using the minimum number of carry-save adders, giving the same speed as a Wallace tree with less hardware. It is split over 2 cycles with a carry-save latch, and the final addition uses the Kogge-Stone adder.



Divider

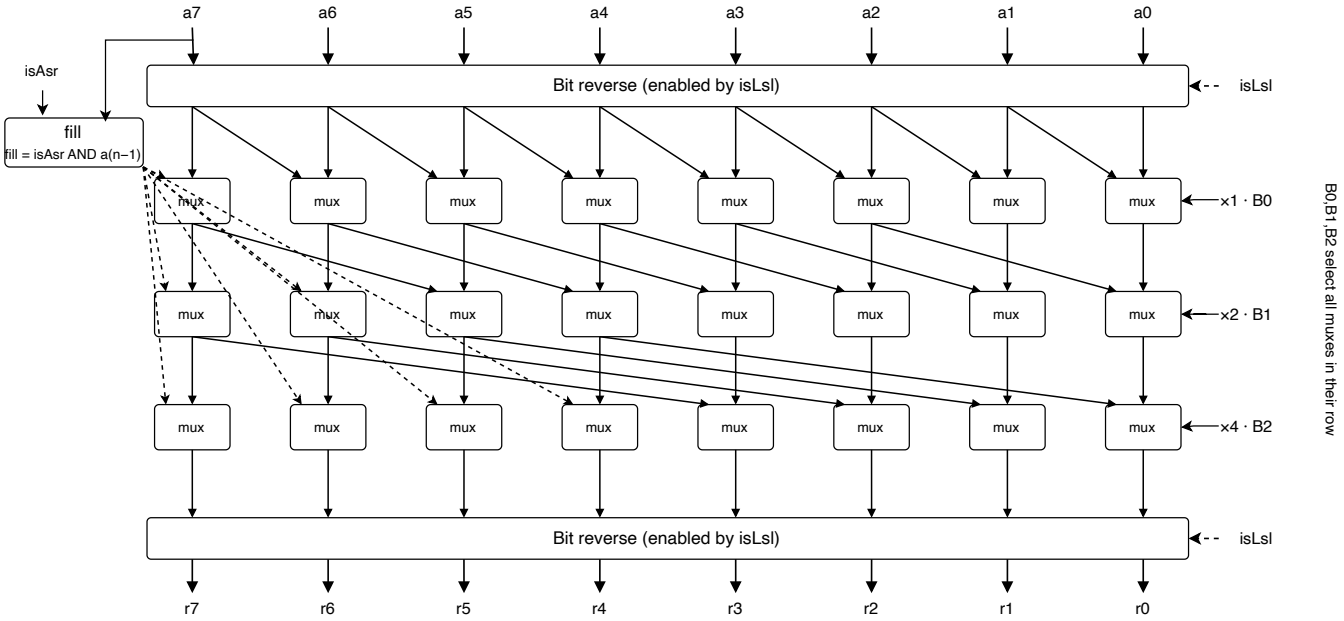
Implemented using the non-restoring division algorithm, shared by div and mod. Unlike restoring division, it never undoes a wrong subtraction; it corrects it with an addition in the next step, so each of the 32 steps needs only one add/subtract. It runs on magnitudes with a final sign fix, takes 32 cycles (aluBusy stalls the pipeline), and raises divByZero when B = 0.



Shifter

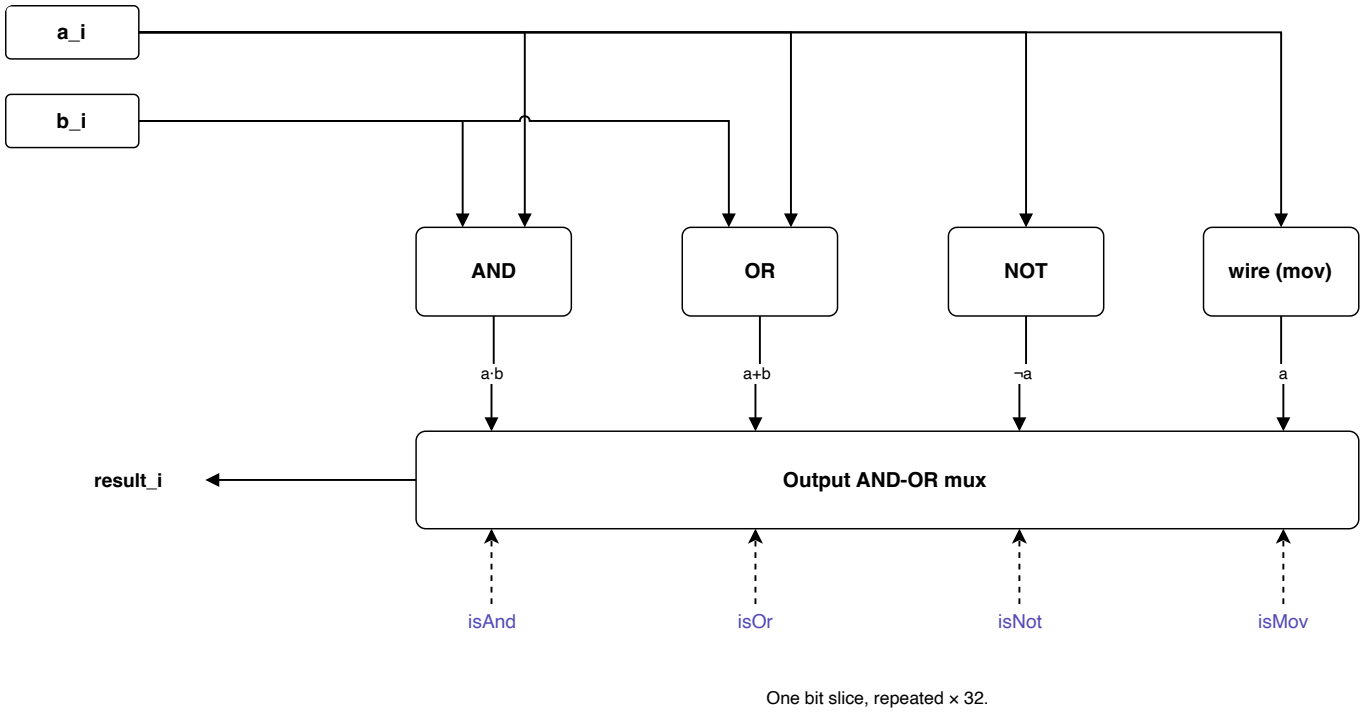
Implemented as a logarithmic barrel shifter, which shifts by any amount (0–31) in a single cycle using 5 layers of 2:1 muxes (shift by 1, 2, 4, 8, 16), each controlled by one bit of the shift amount. A single right-shifting structure handles all three shifts: asr fills vacated bits with the sign bit, and lsl reuses the same

hardware by reversing the bits before and after the shift.



Logical Unit + Mov

Implemented as a bitwise logic slice repeated 32 times. AND and OR take both operands, while NOT and mov use only A (not rd, rs1; mov rd, imm). All four results are computed in parallel every cycle, and the output mux keeps only the one selected by isAnd, isOr, isNot or isMov. This needs just one gate level, so it finishes in a single cycle.

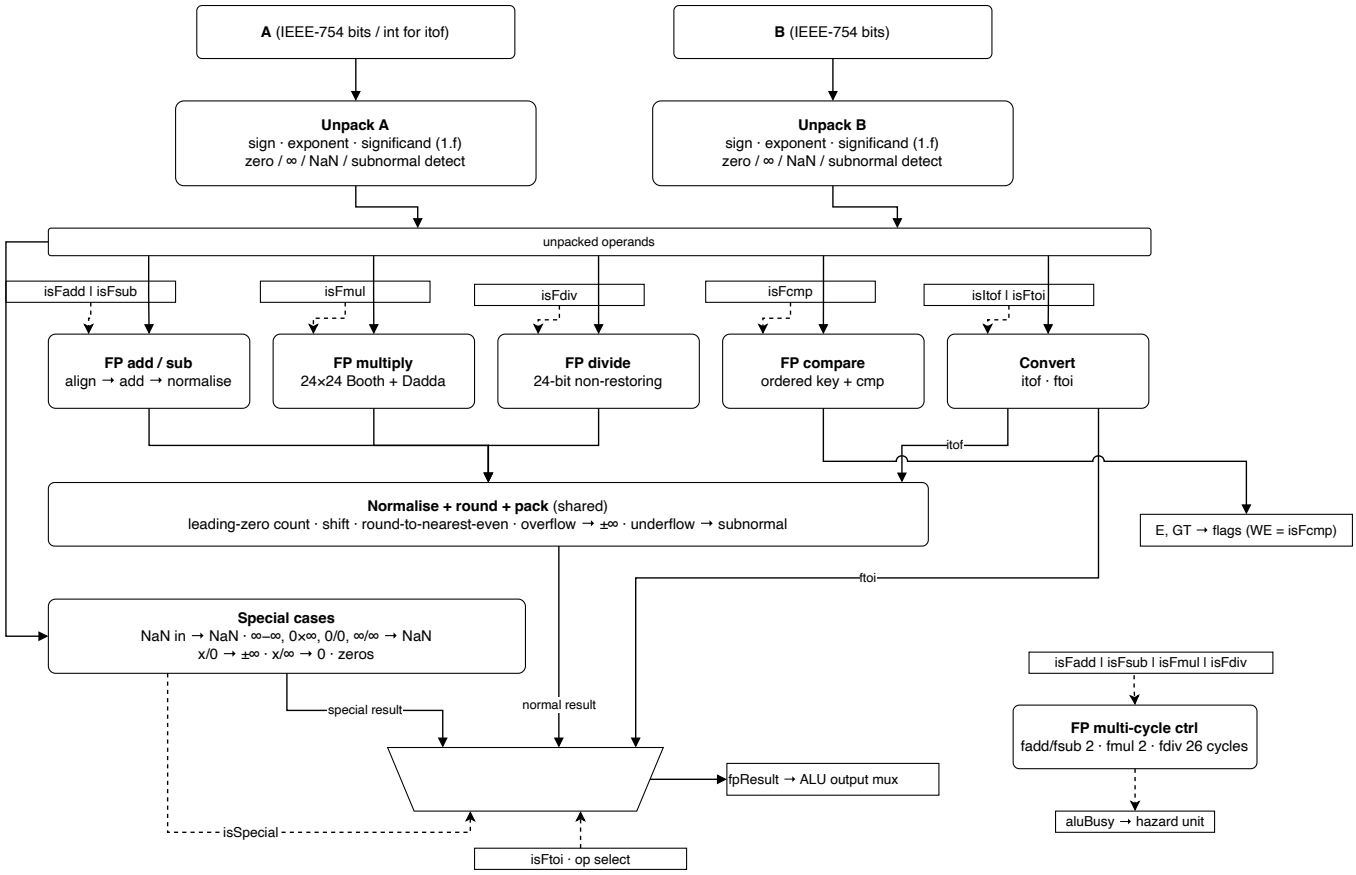


One bit slice, repeated × 32.

FPU Overview

Each operand is split into into sign, exponent and mantissa, then sent to the

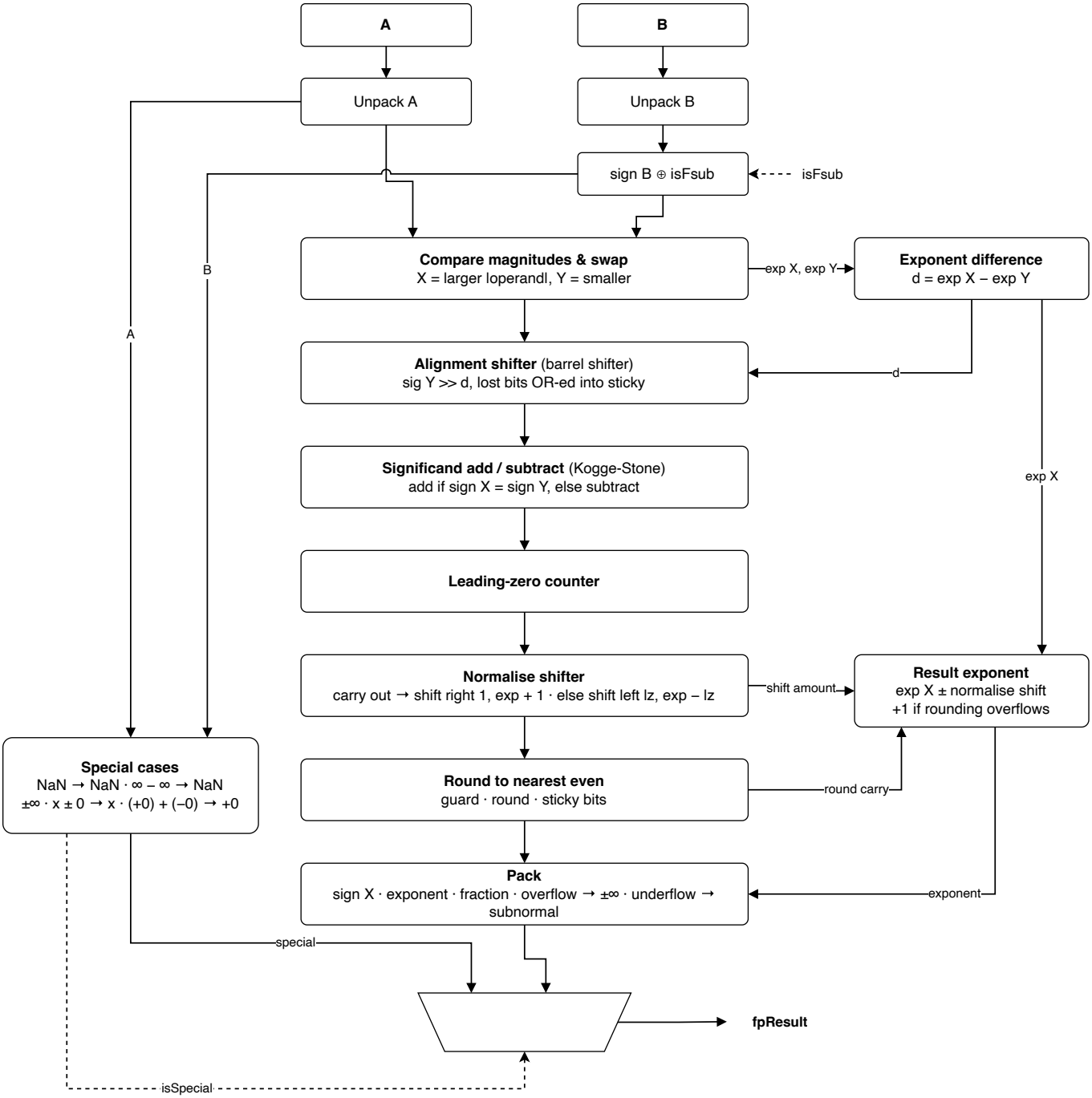
selected unit . A shared stage normalises, rounds to nearest even, and packs the result, while a separate special-case block handles NaN, infinity and zero. The FPU reuses the integer adder, Booth multiplier, divider and barrel shifter for the mantissa arithmetic.



FP Adder

The unit splits A and B into sign, exponent and significand. For fsub, it inverts the sign of B. The unit then compares the two operands. X is the larger operand and Y is the smaller operand. The barrel shifter moves the significand of Y to the right by the exponent difference. The Kogge-Stone adder then adds or subtracts the two significands. The normalise shifter moves the result to the form 1.f and adjusts the exponent. The unit rounds the result to the nearest even value and packs it. A separate block finds special inputs, for example NaN, infinity and

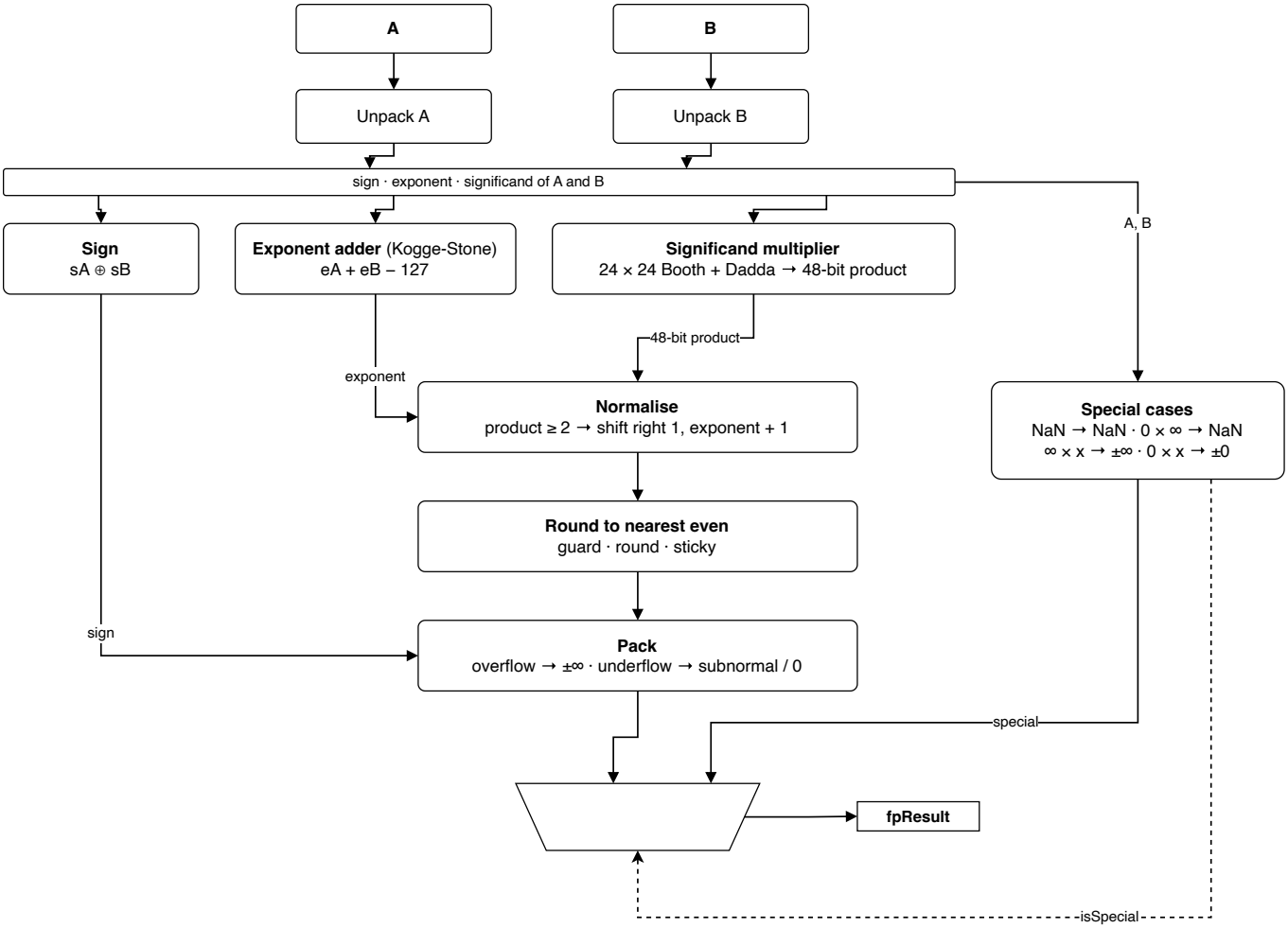
zero. For these inputs, the mux selects the special result.



FP Multiplier

this splits A and B into sign, exponent and significand. The sign of the result is sA XOR sB. The Kogge-Stone adder adds the two exponents and subtracts the bias of 127. The Booth and Dadda multiplier multiplies the two 24-bit significands and makes a 48-bit product. If the product is 2 or more, the normalise step shifts it right by 1 and adds 1 to the exponent. The unit then rounds the result to the nearest even value and packs it. A separate block finds special inputs, for example NaN, infinity and zero. For these inputs, the mux

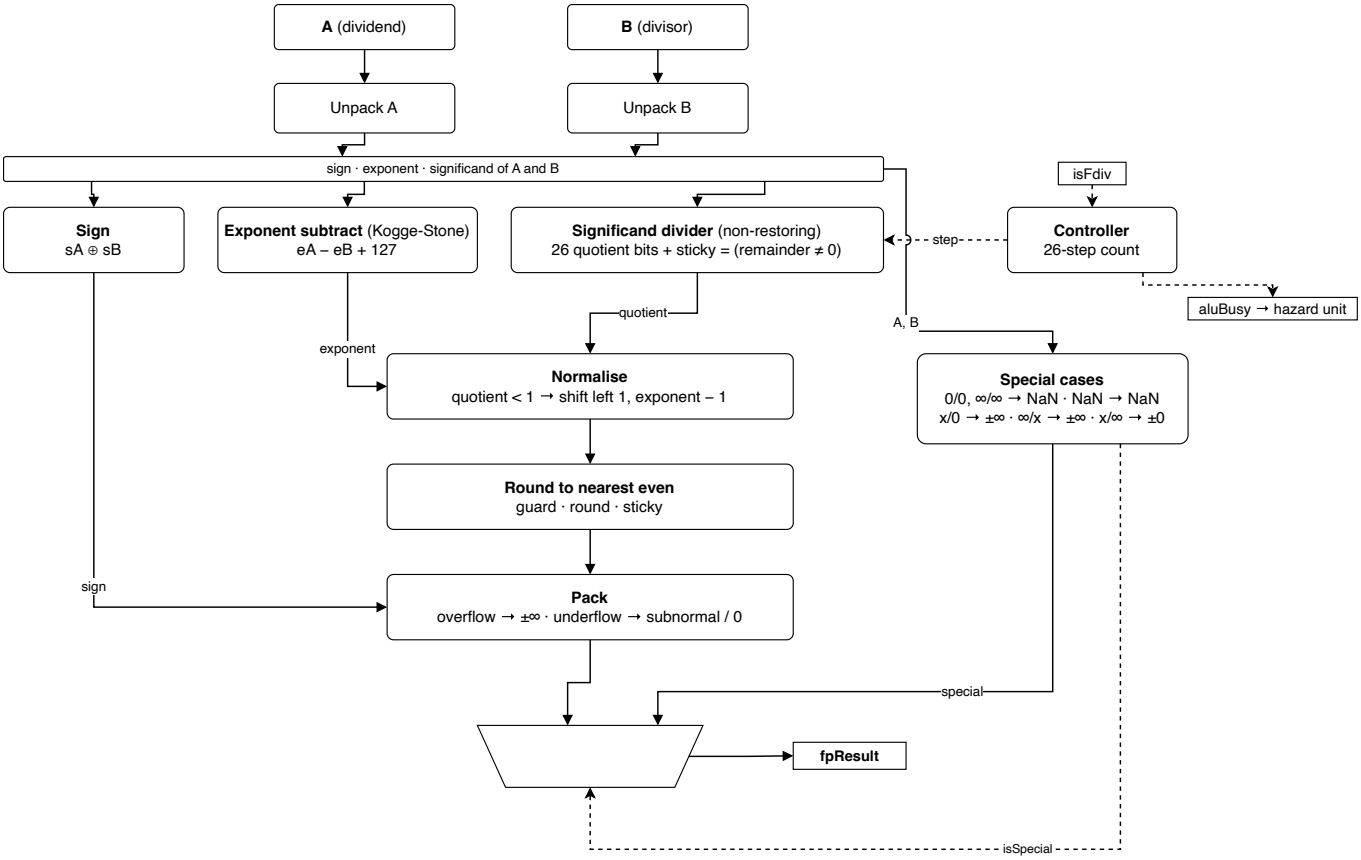
selects the special result. The operation takes 2 cycles.



FP Divider

this splits A and B into sign, exponent and significand. The sign of the result is sA XOR sB. The Kogge-Stone adder subtracts the exponent of B from the exponent of A and adds the bias of 127. The non-restoring divider divides the two significands. It makes 26 quotient bits. If the remainder is not zero, it sets the sticky bit. If the quotient is less than 1, the normalise step shifts it left by 1 and subtracts 1 from the exponent. The unit then rounds the result to the nearest even value and packs it. A separate block finds special inputs, for example 0/0, x/0 and infinity. For these inputs, the mux selects the special result. The

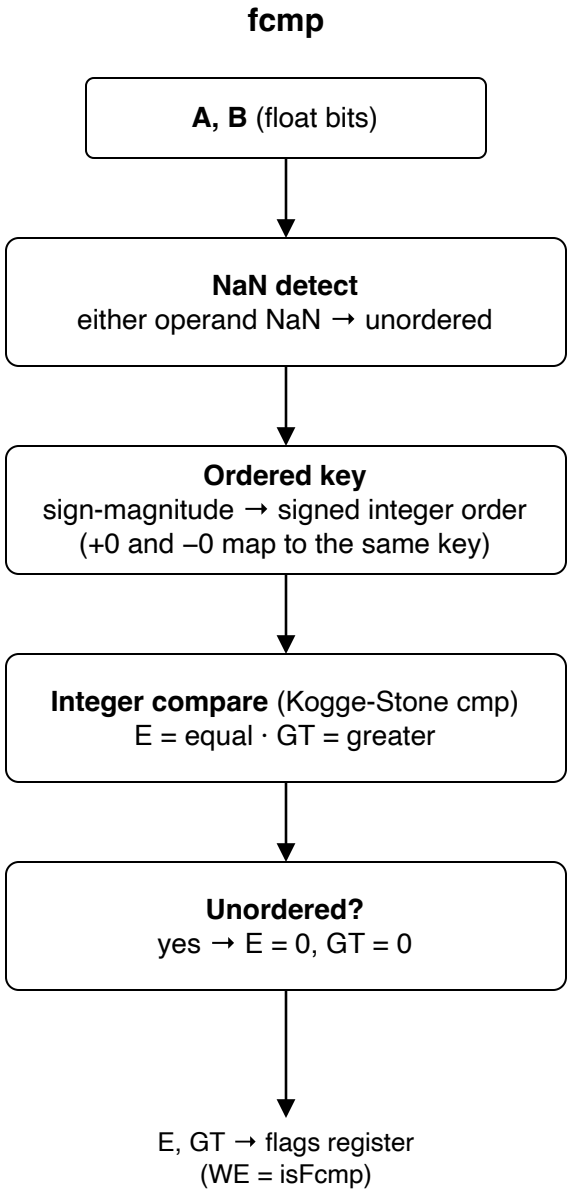
controller counts the 26 steps and sets aluBusy.



FPU fcmp

fcmp compares two floats. If one operand is NaN, the result is unordered and E and GT are 0. Otherwise, the unit changes each operand into an ordered integer key, and the Kogge-Stone adder compares the keys. The unit writes E and GT

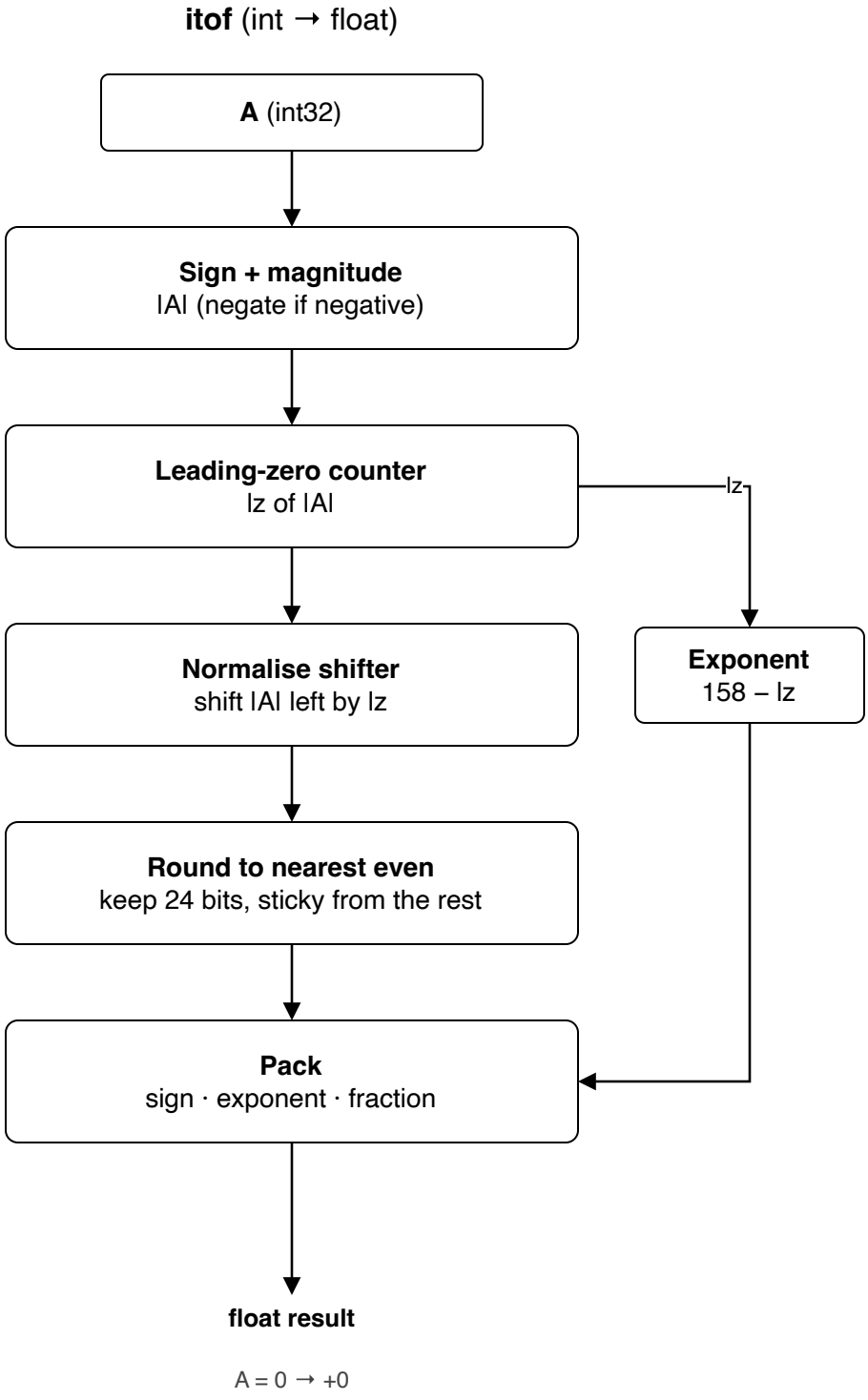
to the flags register.



FPU itof

itof changes a 32-bit integer to a float. The unit gets the sign and the magnitude of A. It counts the leading zeros and shifts the magnitude left. The exponent is

158 minus the leading-zero count. The unit then rounds the result and packs it.

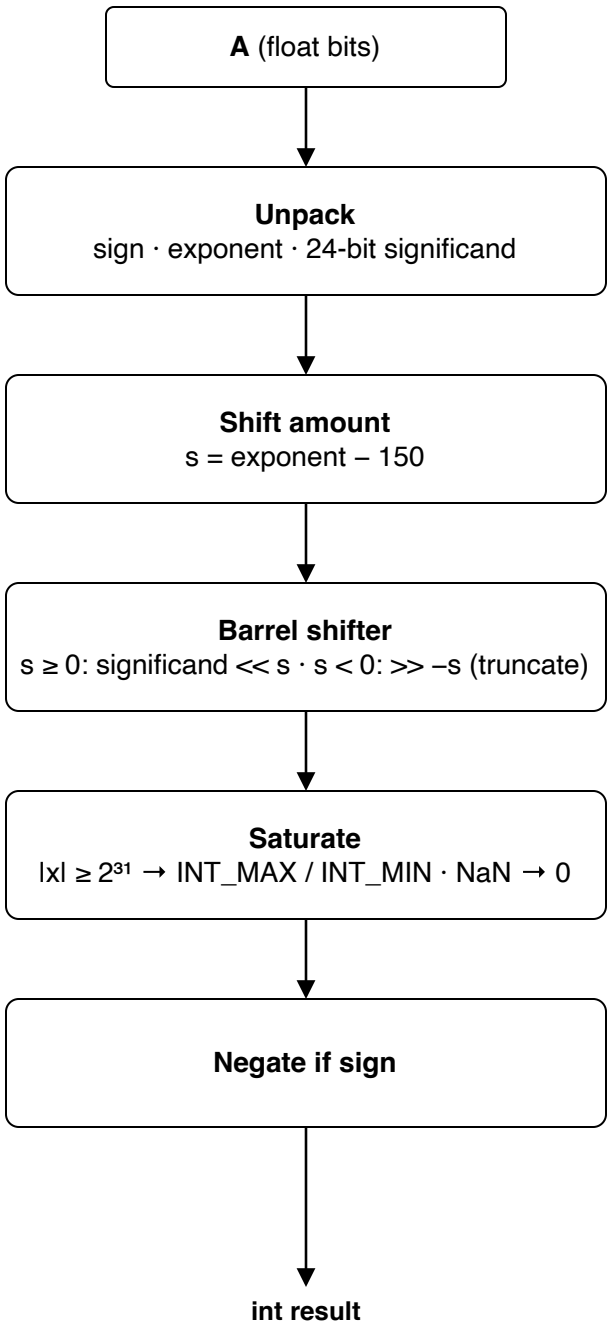


FPU ftoi

ftoi changes a float to a 32-bit integer. The unit unpacks A. The barrel shifter shifts the significand by the exponent minus 150 and removes the fraction. If the value is too large, the unit sets the maximum or minimum integer. If A is

negative, the unit negates the result.

ftoi (float → int)



rounds toward zero, like C